

Online Assessment Prep Guide

Java for Python Programmers: Problem Solving, AI-Assisted Debugging, AI-Assisted Feature Development and Prompt Engineering

Round	Task	Time
2. Problem Solving	1 coding problem (Java)	~20 min
3. AI-Assisted Debugging	1 debugging problem	~20 min
4. AI-Assisted Feature Development	1 feature task in a provided app	~45 min
5. Prompt Engineering	1 prompt-writing task	~15-20 min
Total		~100-105 min

NOTE: Round 1 (English) and Round 6 (Cognitive) are not covered here. This guide is built only for the four technical rounds above. Java is assumed for rounds 2 and 3; for round 4 use whatever stack the platform gives you.

Tonight's study plan (be realistic, sleep matters)

Block	What to do	Time
A. Java syntax	Section 2: skeleton, I/O, types, String/StringBuilder, arrays, ArrayList, HashMap, HashSet, Deque, PriorityQueue. Type out the snippets, do not just read.	~90 min
B. Core patterns	Section 3.1-3.7: arrays/strings, HashMap, stack, two pointers, sliding window, binary search. Re-write 5 solutions from memory.	~120 min
C. Recursion, trees, graphs, DP	Sections 3.8-3.12. Focus on BFS/DFS, level order, height, number of islands, 1D DP.	~90 min
D. AI rounds	Sections 5, 6, 7. Read the workflows and memorise the prompt framework.	~45 min
E. Final skim	Section 8 checklist + Java bug cheat sheet. Then stop and sleep.	~20 min

TIP: If you have less time: do A, then arrays/strings + HashMap + two pointers + sliding window + binary search, then sections 5-7. Trees/graphs/DP matter, but a 20-minute round is far more likely to be solved by core patterns plus clean Java.

Contents

1. The 20-Minute Problem Solving Strategy	3
2. Java Syntax for Python Programmers	4
3. Core Patterns with Java Templates	9
4. Practice Problems and Warm-ups	23
5. AI-Assisted Debugging Round (~20 min)	25
6. AI-Assisted Feature Development (~45 min)	28
7. Prompt Engineering Round (~15-20 min)	30
8. Final Checklist and Cheat Sheet	32

1. The 20-Minute Problem Solving Strategy

You have about 20 minutes for one problem. Most points come from a **correct, clean, compiling** solution, so the process matters as much as the algorithm.

Minutes	Do this
0-3	Read the statement twice. Write input/output format and constraints. Work through the sample by hand. Note edge cases (empty, one element, duplicates, negatives, large values).
3-6	State the brute force and its complexity. Look at constraints to find the target complexity. Match the pattern (table below). Decide data structures.
6-15	Code the solution. Write the skeleton and signature first, then fill in. Use clear names. Do not optimise early; get it compiling.
15-19	Dry-run the sample and 2-3 edge cases mentally. Check loop bounds and overflow.
19-20	Submit. If stuck at minute 12 with nothing, submit the brute force; partial test cases usually still earn marks.

Constraints tell you the target complexity

n (input size)	Acceptable complexity	Typical technique
$n \leq 10-12$	$O(n!)$ / $O(2^n)$	Backtracking, brute force
$n \leq 20-25$	$O(2^n)$	Subsets, bitmask
$n \leq 500$	$O(n^3)$	Triple loop, Floyd-Warshall
$n \leq 5,000$	$O(n^2)$	Nested loops, 2D DP
$n \leq 100,000 - 1,000,000$	$O(n \log n)$ or $O(n)$	Sort, heap, binary search, hash, two pointers
$n \leq 10^9$ or more	$O(\log n)$ or $O(1)$	Binary search, math formula

Keyword to technique

If the problem says...	Think...
"pair / two numbers sum to target", "find duplicate", "count frequency"	HashMap / HashSet
"sorted array", "find in sorted", "minimum X such that"	Binary search (including search on answer)
"longest/shortest subarray or substring with condition", "window of size k"	Sliding window
"sorted + pair", "palindrome", "in-place remove/partition"	Two pointers
"matching brackets", "next greater", "undo", "nested"	Stack
"level by level", "shortest path (unweighted)", "minimum steps"	BFS with Queue
"all combinations / subsets / permutations"	Backtracking (recursion)
"number of ways", "min cost", "max profit", overlapping subproblems	Dynamic programming
"connected components", "islands", "reachable"	DFS / BFS / Union-Find
"k largest / smallest", "top k", "merge k"	PriorityQueue (heap)
"intervals", "meetings"	Sort by start, then sweep
"order of tasks with dependencies"	Topological sort

WATCH OUT: Do not spend 10 minutes hunting for the optimal solution. Write the brute force, get the sample passing, then improve if time remains.

2. Java Syntax for Python Programmers

Java is the same logic as Python with **types, semicolons, braces and more ceremony**. Once you know the 10 translations in this section you can write any solution.

2.1 Program skeleton and input

PYTHON (what you know)

```
n = int(input())
a = list(map(int, input().split()))
print(sum(a))
```

JAVA (what to write tomorrow)

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        long sum = 0;
        for (int x : a) sum += x;
        System.out.println(sum);
    }
}
```

JAVA: fast input

```
// Faster input (use when n is large, ~10^5 or more)
public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    int n = Integer.parseInt(br.readLine().trim());
    StringTokenizer st = new StringTokenizer(br.readLine());
    int[] a = new int[n];
    for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());
}
// Scanner extras: sc.nextLine() (whole line), sc.next() (one word), sc.nextLong(), sc.hasNext()
// After sc.nextInt(), call sc.nextLine() once to consume the leftover newline before reading a line.
```

NOTE: Most platforms give you a ready function signature such as `public int solve(int[] nums)` inside `class Solution`. In that case you do not write main or read input. Just fill in the method and keep the signature unchanged.

2.2 Types and the traps Python hides from you

Java type	Notes	Python equivalent
<code>int</code>	32-bit, range about ± 2.147 billion (2147483647). Overflows silently .	<code>int</code> (unbounded)
<code>long</code>	64-bit, literal needs <code>L</code> suffix: <code>1000000007L</code> . Use for sums and products.	<code>int</code>
<code>double</code>	64-bit float. <code>float</code> is rarely needed.	<code>float</code>
<code>boolean</code>	<code>true</code> / <code>false</code> (lowercase)	<code>True</code> / <code>False</code>
<code>char</code>	Single quotes <code>'a'</code> . Behaves like a number: <code>c - 'a'</code> gives 0..25.	length-1 <code>str</code>
<code>String</code>	Double quotes. Immutable , compare with <code>.equals()</code> .	<code>str</code>
<code>int[]</code>	Fixed-size array. <code>a.length</code> (no parentheses).	fixed-size <code>list</code>
<code>ArrayList<Integer></code>	Resizable list. <code>list.size()</code> .	<code>list</code>

JAVA: numeric traps

```
int x = 7 / 2;           // 3      (integer division, truncates toward zero; Python 7//2 = 3 too)
int y = -7 / 2;          // -3     (Python -7//2 = -4 !)
int m = -7 % 3;          // -1     (Python -7 % 3 = 2 !) -> use Math.floorMod(-7, 3) = 2
double d = 7 / 2.0;      // 3.5    (one side must be double)
int big = Integer.MAX_VALUE + 1; // overflow -> -2147483648
long ok = 1L * a * b;     // promote to long BEFORE multiplying
long mod = 1_000_000_007L;
int mid = lo + (hi - lo) / 2; // safe midpoint
char c = 's'; int idx = c - 'a'; // 18
char next = (char) (c + 1); // 't' (cast back to char)
```

WATCH OUT: `==` on Strings and boxed types compares references, not values. Use `s1.equals(s2)` for Strings and `a.equals(b)` for Integer objects (for example values taken from a `Map<String,Integer>`). Integer `a=1000, b=1000; a==b` is **false**.

2.3 Control flow

PYTHON (what you know)

```
for i in range(n): ...
for i in range(1, n+1): ...
for i in range(n-1, -1, -1): ...
for x in arr: ...
for i, x in enumerate(arr): ...
while cond: ...
if a and not b or c: ...
x = a if cond else b
```

JAVA (what to write tomorrow)

```
for (int i = 0; i < n; i++) { ... }
for (int i = 1; i <= n; i++) { ... }
for (int i = n - 1; i >= 0; i--) { ... }
for (int x : arr) { ... }
for (int i = 0; i < arr.length; i++) { int x = arr[i]; ... }
while (cond) { ... }
if (a && !b || c) { ... }
int x = cond ? a : b;
// switch, break, continue work as in other languages; "elif" is "else if"
```

2.4 Strings, StringBuilder and chars

Task	Python	Java
Length	<code>len(s)</code>	<code>s.length()</code>
Char at i	<code>s[i]</code>	<code>s.charAt(i)</code>
Substring [i, j)	<code>s[i:j]</code>	<code>s.substring(i, j)</code>
Equality	<code>s == t</code>	<code>s.equals(t)</code>
Lexicographic compare	<code>s < t</code>	<code>s.compareTo(t)</code> (negative, 0, positive)
Contains / find	<code>t in s, s.find(t)</code>	<code>s.contains(t), s.indexOf(t)</code> (-1 if missing)
Split / join	<code>s.split(), " ".join(l)</code>	<code>s.split(" "), String.join(" ", list)</code>
Upper / lower / strip	<code>upper() lower() strip()</code>	<code>toUpperCase() toLowerCase() trim()</code>
Reverse	<code>s[::-1]</code>	<code>new StringBuilder(s).reverse().toString()</code>
To char list and back	<code>list(s), "".join(l)</code>	<code>s.toCharArray(), new String(chars) or String.valueOf(chars)</code>
Int to string / back	<code>str(n), int(s)</code>	<code>String.valueOf(n), Integer.parseInt(s)</code>
Char tests	<code>c.isdigit(), c.isalpha()</code>	<code>Character.isDigit(c), isLetter, isLetterOrDigit, isUpperCase</code>
Repeat	<code>"ab" * 3</code>	<code>"ab".repeat(3)</code>
Replace	<code>s.replace("a", "b")</code>	<code>s.replace("a", "b")</code> (replaceAll uses regex)

JAVA: StringBuilder and character counting

```
// Never build strings with += in a loop (O(n^2)). Use StringBuilder.
StringBuilder sb = new StringBuilder();
for (char c : s.toCharArray()) {
    if (Character.isLetter(c)) sb.append(Character.toLowerCase(c));
}
String cleaned = sb.toString();
sb.append(5).append(' ').append("text"); // chain appends
sb.reverse(); sb.insert(0, 'x'); sb.setCharAt(2, 'z'); sb.deleteCharAt(sb.length() - 1);
// Frequency of lowercase letters (replaces Counter / dict for letters)
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++;
```

2.5 Arrays

PYTHON (what you know)

```
a = [0] * n
grid = [[0] * m for _ in range(r)]
a.sort(); a.sort(reverse=True)
b = a[:] # copy
print(a)
max(a); min(a); sum(a)
```

JAVA (what to write tomorrow)

```
int[] a = new int[n]; // filled with 0
int[][] grid = new int[r][m]; // r rows, m cols; grid.length = r, grid[0].length = m
Arrays.sort(a); // ascending, in place
Integer[] boxed = {3, 1, 2};
Arrays.sort(boxed, Collections.reverseOrder()); // descending needs Integer[], not int[]
int[] b = Arrays.copyOf(a, a.length); // copy; also Arrays.copyOfRange(a, from, to)
System.out.println(Arrays.toString(a)); // [1, 2, 3] (2D: Arrays.deepToString(grid))
Arrays.fill(a, -1);
int[] lit = {5, 3, 9}; // literal
int mx = Arrays.stream(a).max().getAsInt(); // or a manual loop (simpler, less error-prone)
long s = Arrays.stream(a).asLongStream().sum();
```

WATCH OUT: Printing an array directly (`System.out.println(a)`) prints a memory address like `[I@1b6d3586`. Always use `Arrays.toString(a)`.

2.6 Collections: the Python to Java map

Python	Java	Key methods
list	<code>ArrayList<Integer></code>	<code>add(x)</code> , <code>get(i)</code> , <code>set(i,x)</code> , <code>size()</code> , <code>remove(int index)</code> , <code>contains(x)</code> , <code>isEmpty()</code> , <code>clear()</code>
dict	<code>HashMap<K,V></code>	<code>put</code> , <code>get</code> , <code>getOrDefault(k,d)</code> , <code>containsKey</code> , <code>remove</code> , <code>keySet()</code> , <code>values()</code> , <code>entrySet()</code> , <code>merge</code>
sorted dict	<code>TreeMap<K,V></code>	same + <code>firstKey</code> , <code>lastKey</code> , <code>floorKey</code> , <code>ceilingKey</code>
set	<code>HashSet<T></code>	<code>add</code> (returns false if present), <code>contains</code> , <code>remove</code> , <code>size</code>
sorted set	<code>TreeSet<T></code>	<code>first</code> , <code>last</code> , <code>floor</code> , <code>ceiling</code>
stack (list)	<code>Deque<T> st = new ArrayDeque<>()</code>	<code>push</code> , <code>pop</code> , <code>peek</code> , <code>isEmpty</code> (do not use legacy <code>Stack</code>)
queue (deque)	<code>Queue<T> q = new ArrayDeque<>()</code>	<code>offer</code> , <code>poll</code> , <code>peek</code> , <code>size</code> , <code>isEmpty</code>
deque	<code>Deque<T></code>	<code>addFirst</code> , <code>addLast</code> , <code>pollFirst</code> , <code>pollLast</code> , <code>peekFirst</code> , <code>peekLast</code>
heapq	<code>PriorityQueue<T></code>	<code>offer</code> , <code>poll</code> , <code>peek</code> (min-heap by default)
Counter	<code>HashMap<T,Integer> + merge</code>	<code>map.merge(x, 1, Integer::sum)</code>
<code>defaultdict(list)</code>	<code>computeIfAbsent</code>	<code>map.computeIfAbsent(k, z -> new ArrayList<>()).add(v)</code>
tuple / pair	<code>int[]{a,b}</code> or small class	Use <code>int[]</code> in queues and heaps

JAVA: collections cookbook

```
// Generics need wrapper types: Integer, Long, Character, Double, Boolean (never int, char).
List<Integer> list = new ArrayList<>();
list.add(5); list.add(7);
list.remove(Integer.valueOf(7)); // remove by VALUE. list.remove(1) removes by INDEX!
Collections.sort(list); // ascending
Collections.reverse(list);
int[] arr = list.stream().mapToInt(Integer::intValue).toArray(); // list -> int[]
List<Integer> fromArr = new ArrayList<>(); for (int x : arr) fromArr.add(x); // int[] -> list

Map<String, Integer> count = new HashMap<>();
for (String w : words) count.put(w, count.getOrDefault(w, 0) + 1); // or count.merge(w, 1,
Integer::sum)
for (Map.Entry<String, Integer> e : count.entrySet()) {
    String key = e.getKey(); int val = e.getValue();
}
for (String k : count.keySet()) { ... }

Set<Integer> seen = new HashSet<>();
if (!seen.add(x)) { /* x was already present -> duplicate found */ }

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); int top = stack.peek(); stack.pop();

Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[]{r, c}); int[] cur = q.poll();

PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1])); // by 2nd value
```

WATCH OUT: Do not call `stack.peek()` / `pop()` / `q.poll()` without checking `isEmpty()` first. `pop()` on an empty `ArrayDeque` throws `NoSuchElementException`; `poll()` returns null.

2.7 Sorting with comparators

PYTHON (what you know)

```
people.sort(key=lambda p: (p[1], -p[0]))
intervals.sort(key=lambda x: x[0])
words.sort(key=len)
```

JAVA (what to write tomorrow)

```
Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0])); // 2D array by first column
Arrays.sort(people, (a, b) -> a[1] != b[1] ? Integer.compare(a[1], b[1]) : Integer.compare(b[0], a[0]));
Arrays.sort(words, (a, b) -> a.length() - b.length()); // String[]
Collections.sort(list, (a, b) -> b - a); // List<Integer>, descending
list.sort(Comparator.comparingInt(String::length)); // List<String>
```

TIP: Prefer `Integer.compare(a, b)` over `a - b` in comparators. Subtraction can overflow for large or negative values.

2.8 Methods, classes and static

JAVA: methods and a simple class

```
public class Main {
    // helper methods called from static main must be static too
    static int add(int a, int b) { return a + b; }

    static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }

    // arrays and objects are passed by reference (changes are visible to caller);
    // int, long, char, boolean are passed by value (changes are not visible)

    static class Node { // nested helper class
        int val; Node next;
        Node(int val) { this.val = val; }
    }

    static int[] memo = new int[100]; // shared state: make it static
}

// Python: def add(a, b): return a + b    class Node: def __init__(self, val): self.val = val;
// self.next = None
```

Math helpers: `Math.max`, `Math.min`, `Math.abs`, `Math.pow(a, b)` (returns double, cast with `(int)` or `(long)`), `Math.sqrt`, `Math.floorMod`, `Integer.MAX_VALUE`, `Integer.MIN_VALUE`, `Long.MAX_VALUE`. **Output:** `System.out.println(x)`, `System.out.print(x)`, `System.out.printf("%d %.2f%n", n, d)`.

2.9 Quick conversion method: Python solution to Java

1. Write the Python logic first (you think fastest in it). Keep it in comments or on paper.
2. Decide the Java type of every variable. Use `long` if a sum or product can pass 2 billion.
3. Replace `list` with `int[]` if the size is fixed, otherwise `ArrayList<Integer>`; `dict` becomes `HashMap`, `set` becomes `HashSet`, stack becomes `ArrayDeque`.
4. Replace slicing, negative indexes and `//` with explicit index math. Python `a[-1]` is `a[a.length-1]`.
5. Add semicolons, braces, parentheses around conditions, and `&&`, `||`, `!` for and/or/not.
6. Compile mentally: every variable declared with a type, every non-void method returns on all paths.

Worked example: first unique character

PYTHON (what you know)

```
def first_unique(s):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    for i, ch in enumerate(s):
        if count[ch] == 1:
            return i
    return -1
```

JAVA (what to write tomorrow)

```
static int firstUnique(String s) {
    Map<Character, Integer> count = new HashMap<>();
    for (char ch : s.toCharArray()) {
        count.put(ch, count.getOrDefault(ch, 0) + 1);
    }
    for (int i = 0; i < s.length(); i++) {
        if (count.get(s.charAt(i)) == 1) return i;
    }
    return -1;
}
```

3. Core Patterns with Java Templates

Each pattern: when to use it, a Python mental model where useful, and a tested Java template. All methods are `static` so you can paste them inside `public class Main` (or adapt to `class Solution`).

3.1 Arrays and Strings

- **Prefix sum:** `pre[i+1] = pre[i] + a[i]`; sum of range `[l, r]` = `pre[r+1] - pre[l]`.
- **Kadane (max subarray):** at each element either extend the running sum or restart.
- **In-place tricks:** swap with two indexes, write pointer for filtering (move zeros, remove element).
- **Strings:** palindrome (two pointers), anagram (count array of 26), reverse words (split + join).

Kadane: maximum subarray sum

PYTHON (what you know)

```
def max_subarray(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

JAVA (what to write tomorrow)

```
static int maxSubArray(int[] a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

JAVA: array and string templates

```
static void moveZeroes(int[] a) {           // stable, in place, O(n)
    int w = 0;                             // write pointer
    for (int r = 0; r < a.length; r++) {
        if (a[r] != 0) { int t = a[w]; a[w] = a[r]; a[r] = t; w++; }
    }
}

static boolean isPalindrome(String s) {     // ignore non-alphanumerics and case
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r))) return false;
        l++; r--;
    }
    return true;
}

static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] c = new int[26];
    for (int i = 0; i < s.length(); i++) { c[s.charAt(i) - 'a']++; c[t.charAt(i) - 'a']--; }
    for (int x : c) if (x != 0) return false;
    return true;
}

static void rotate(int[] a, int k) {        // rotate right by k using three reversals
    int n = a.length; k %= n;
    reverse(a, 0, n - 1); reverse(a, 0, k - 1); reverse(a, k, n - 1);
}

static void reverse(int[] a, int i, int j) {
    while (i < j) { int t = a[i]; a[i++] = a[j]; a[j--] = t; }
}

static int[] prefixSums(int[] a) {
    int[] pre = new int[a.length + 1];
    for (int i = 0; i < a.length; i++) pre[i + 1] = pre[i] + a[i];
    return pre;                             // range sum [l..r] = pre[r+1] - pre[l]
}
```

3.2 HashMap and HashSet

Use when you need $O(1)$ lookup: "have I seen this?", "how many times?", "what index?". Trade memory for time: turns many $O(n^2)$ loops into $O(n)$.

Two Sum

PYTHON (what you know)

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
```

JAVA (what to write tomorrow)

```
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) return new int[]{seen.get(need), i};
        seen.put(nums[i], i);
    }
    return new int[]{-1, -1};
}
```

JAVA: HashMap / HashSet templates

// Subarray sum equals K (prefix sum + map). Handles negatives, unlike sliding window.

```
static int subarraySum(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    cnt.put(0, 1); // empty prefix
    int sum = 0, res = 0;
    for (int x : a) {
        sum += x;
        res += cnt.containsKey(sum - k) ? cnt.get(sum - k) : 0;
        cnt.put(sum, cnt.getOrDefault(sum, 0) + 1);
    }
    return res;
}
```

// Group anagrams: key = sorted characters

```
static List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> m = new HashMap<>();
    for (String s : strs) {
        char[] c = s.toCharArray(); Arrays.sort(c);
        m.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(m.values());
}
```

// Longest consecutive sequence, $O(n)$ with HashSet

```
static int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {
        if (!set.contains(x - 1)) { // only start at the beginning of a run
            int len = 1, y = x;
            while (set.contains(y + 1)) { y++; len++; }
            best = Math.max(best, len);
        }
    }
    return best;
}
```

// Contains duplicate / first repeated element

```
static boolean hasDuplicate(int[] a) {
    Set<Integer> s = new HashSet<>();
    for (int x : a) if (!s.add(x)) return true;
    return false;
}
```

TIP: Frequency of letters only? `int[26]` is faster and simpler than a HashMap. Frequency of arbitrary numbers or strings? HashMap with `merge`.

3.3 Stack and Queue

- **Stack (LIFO):** matching brackets, evaluate expressions, next greater element, undo, DFS iteratively.

- **Queue (FIFO):** BFS, level order, shortest path in unweighted graph/grid, simulation.
- **Monotonic stack:** keep stack ordered to find next greater/smaller in O(n).
- **Deque:** sliding window maximum (keep indexes of decreasing values).

Valid parentheses

PYTHON (what you know)

```
def is_valid(s):
    pair = {'(': ')', '[': ']', '{': '}'
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]: return False
        else:
            st.append(c)
    return not st
```

JAVA (what to write tomorrow)

```
static boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.isEmpty()) return false;
            char t = st.pop();
            if ((c == ')' && t != '(') || (c == ']' && t != '[') || (c == '}' && t != '{')) return
                false;
        }
    }
    return st.isEmpty();
}
```

JAVA: stack and deque templates

```
// Next greater element to the right (monotonic stack of indexes), -1 if none
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    Arrays.fill(res, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] < a[i]) res[st.pop()] = a[i];
        st.push(i);
    }
    return res;
}

// Daily temperatures: days until a warmer day
static int[] dailyTemperatures(int[] t) {
    int[] res = new int[t.length];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < t.length; i++) {
        while (!st.isEmpty() && t[st.peek()] < t[i]) { int j = st.pop(); res[j] = i - j; }
        st.push(i);
    }
    return res;
}

// Sliding window maximum with a deque of indexes (values decreasing front to back)
static int[] maxSliding(int[] a, int k) {
    int n = a.length; int[] res = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst(); // out of window
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast(); // smaller useless
        dq.offerLast(i);
        if (i >= k - 1) res[i - k + 1] = a[dq.peekFirst()];
    }
    return res;
}

// Evaluate Reverse Polish Notation
static int evalRPN(String[] tokens) {
    Deque<Integer> st = new ArrayDeque<>();
    for (String t : tokens) {
        if (t.equals("+") || t.equals("-") || t.equals("*") || t.equals("/")) {
            int b = st.pop(), a = st.pop();
            if (t.equals("+")) st.push(a + b);
```

```

        else if (t.equals("-")) st.push(a - b);
        else if (t.equals("*")) st.push(a * b);
        else st.push(a / b);
    } else st.push(Integer.parseInt(t));
}
return st.pop();
}

```

3.4 Two Pointers

Two indexes moving through the data. **Opposite ends** (sorted array, palindrome, container) or **same direction** (fast/slow, write/read pointer). Usually $O(n)$ after an optional sort.

JAVA: two-pointer templates

```

// Two Sum on a SORTED array (1 pass,  $O(1)$  space)
static int[] twoSumSorted(int[] a, int target) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return new int[]{l, r};
        if (s < target) l++; else r--;
    }
    return new int[]{-1, -1};
}

// Remove duplicates from sorted array in place; returns new length
static int removeDuplicates(int[] a) {
    if (a.length == 0) return 0;
    int w = 1;
    for (int r = 1; r < a.length; r++) if (a[r] != a[r - 1]) a[w++] = a[r];
    return w;
}

// Container with most water
static int maxArea(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--; // move the shorter wall
    }
    return best;
}

// 3Sum: sort, fix one, two-pointer the rest, skip duplicates
static List<List<Integer>> threeSum(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < a.length - 2; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        int l = i + 1, r = a.length - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                res.add(Arrays.asList(a[i], a[l], a[r]));
                l++; r--;
                while (l < r && a[l] == a[l - 1]) l++;
                while (l < r && a[r] == a[r + 1]) r--;
            } else if (s < 0) l++; else r--;
        }
    }
    return res;
}

// Merge two sorted arrays
static int[] mergeSorted(int[] x, int[] y) {
    int[] res = new int[x.length + y.length];
    int i = 0, j = 0, k = 0;
    while (i < x.length && j < y.length) res[k++] = x[i] <= y[j] ? x[i++] : y[j++];
    while (i < x.length) res[k++] = x[i++];
    while (j < y.length) res[k++] = y[j++];
    return res;
}

```

3.5 Sliding Window

Fixed window (size k): add the new element, remove the old one. **Variable window**: expand **right** every step; while the window is invalid, shrink **left**. Record the answer when valid.

Longest substring without repeating characters

PYTHON (what you know)

```
def longest_unique(s):
    last, left, best = {}, 0, 0
    for r, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = r
        best = max(best, r - left + 1)
    return best
```

JAVA (what to write tomorrow)

```
static int lengthOfLongestSubstring(String s) {
    int[] last = new int[128];
    Arrays.fill(last, -1); // last seen index of each ASCII char
    int best = 0, left = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (last[c] >= left) left = last[c] + 1;
        last[c] = r;
        best = Math.max(best, r - left + 1);
    }
    return best;
}
```

JAVA: sliding window templates

```
// Fixed window: max sum of any k consecutive elements
static int maxSumK(int[] a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    int best = sum;
    for (int i = k; i < a.length; i++) { sum += a[i] - a[i - k]; best = Math.max(best, sum); }
    return best;
}

// Variable window: shortest subarray with sum >= target (positive numbers)
static int minSubArrayLen(int target, int[] a) {
    int l = 0, sum = 0, best = Integer.MAX_VALUE;
    for (int r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum >= target) { best = Math.min(best, r - l + 1); sum -= a[l++]; }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}

// Longest substring with at most K distinct characters
static int longestKDistinct(String s, int k) {
    Map<Character, Integer> cnt = new HashMap<>();
    int l = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        cnt.merge(s.charAt(r), 1, Integer::sum);
        while (cnt.size() > k) {
            char c = s.charAt(l++);
            if (cnt.merge(c, -1, Integer::sum) == 0) cnt.remove(c);
        }
        best = Math.max(best, r - l + 1);
    }
    return best;
}

// Longest repeating character replacement (replace at most k chars)
static int characterReplacement(String s, int k) {
    int[] cnt = new int[26]; int l = 0, maxF = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        maxF = Math.max(maxF, ++cnt[s.charAt(r) - 'A']);
        while (r - l + 1 - maxF > k) cnt[s.charAt(l++) - 'A']--;
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

NOTE: Sliding window with a shrinking **while** loop only works when numbers are non-negative (or the condition is monotonic). For arrays with negatives use prefix sum + HashMap.

3.6 Binary Search

Requires a **monotonic** property: sorted data, or a yes/no check that flips once. Always compute $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$. Pick ONE loop style and use it consistently.

JAVA: binary search templates

```
// 1) Classic: find target, return index or -1.      Loop: lo <= hi
static int binarySearch(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}

// 2) Lower bound: first index with a[i] >= t (returns a.length if none).  Loop: lo < hi
static int lowerBound(int[] a, int t) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] >= t) hi = mid; else lo = mid + 1;
    }
    return lo;
}
// first/last occurrence: first = lowerBound(a, t); last = lowerBound(a, t + 1) - 1
// count of t = lowerBound(a, t + 1) - lowerBound(a, t)

// 3) Search on the answer: smallest speed/capacity that is feasible
static int minEatingSpeed(int[] piles, int h) {      // Koko eating bananas
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        long hours = 0;
        for (int p : piles) hours += (p + mid - 1) / mid;    // ceil(p / mid)
        if (hours <= h) hi = mid; else lo = mid + 1;        // feasible -> try smaller
    }
    return lo;
}

// 4) Rotated sorted array
static int searchRotated(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[lo] <= a[mid]) {                                // left half sorted
            if (a[lo] <= t && t < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {                                             // right half sorted
            if (a[mid] < t && t <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}

// 5) Integer square root
static int mySqrt(int x) {
    long lo = 0, hi = x;
    while (lo < hi) { long mid = lo + (hi - lo + 1) / 2; if (mid * mid <= x) lo = mid; else hi = mid - 1; }
    return (int) lo;
}
```

TIP: Search on answer recipe: (1) define **feasible(x)**, (2) set lo/hi to the smallest/largest possible answer, (3) if **feasible(mid)** move **hi = mid** else **lo = mid + 1**, (4) answer is lo. Typical words: "minimum largest sum", "minimum days", "maximum minimum distance".

3.7 Linked List (bonus: appears often in debugging tasks)

JAVA: linked list templates (dummy node, slow/fast, reverse)

```
class ListNode { int val; ListNode next; ListNode(int v) { val = v; } }

static ListNode reverseList(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode nxt = cur.next;    // save
        cur.next = prev;           // flip
        prev = cur; cur = nxt;     // advance
    }
    return prev;
}

static ListNode middle(ListNode head) {           // slow/fast pointers
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    return slow;
}

static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

static ListNode mergeTwo(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), t = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { t.next = a; a = a.next; } else { t.next = b; b = b.next; }
        t = t.next;
    }
    t.next = (a != null) ? a : b;
    return dummy.next;
}
```

3.8 Recursion and Backtracking

Every recursive function needs: (1) a **base case**, (2) a smaller subproblem, (3) a trust that the recursive call works. Backtracking = choose, explore, **undo**. Always copy the current list when saving a result: `new ArrayList<>(cur)`.

JAVA: recursion and backtracking

```
static long factorial(int n) { return n <= 1 ? 1 : n * factorial(n - 1); }

static long power(long x, int n) { // fast power O(log n)
    if (n == 0) return 1;
    long half = power(x, n / 2);
    return n % 2 == 0 ? half * half : half * half * x;
}

// Memoised Fibonacci (Python: @lru_cache)
static long[] memo = new long[100];
static long fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}

// Subsets (include / exclude)
static void subsets(int[] a, int i, List<Integer> cur, List<List<Integer>> res) {
    if (i == a.length) { res.add(new ArrayList<>(cur)); return; }
    cur.add(a[i]); subsets(a, i + 1, cur, res); // choose
    cur.remove(cur.size() - 1); subsets(a, i + 1, cur, res); // un-choose
}

// Permutations
static void permute(int[] a, boolean[] used, List<Integer> cur, List<List<Integer>> res) {
    if (cur.size() == a.length) { res.add(new ArrayList<>(cur)); return; }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true; cur.add(a[i]);
        permute(a, used, cur, res);
        cur.remove(cur.size() - 1); used[i] = false;
    }
}

// Combination sum (numbers reusable)
static void comb(int[] a, int start, int rem, List<Integer> cur, List<List<Integer>> res) {
    if (rem == 0) { res.add(new ArrayList<>(cur)); return; }
    for (int i = start; i < a.length; i++) {
        if (a[i] > rem) continue;
        cur.add(a[i]);
        comb(a, i, rem - a[i], cur, res); // i (not i+1) because reuse is allowed
        cur.remove(cur.size() - 1);
    }
}

// Generate valid parentheses
static void gen(int open, int close, int n, StringBuilder sb, List<String> res) {
    if (sb.length() == 2 * n) { res.add(sb.toString()); return; }
    if (open < n) { sb.append('('); gen(open + 1, close, n, sb, res); sb.deleteCharAt(sb.length() - 1); }
    if (close < open) { sb.append(')'); gen(open, close + 1, n, sb, res); sb.deleteCharAt(sb.length() - 1); }
}
```

WATCH OUT: Forgetting to undo (`cur.remove(cur.size()-1)`) or adding `cur` without copying are the two most common backtracking bugs. Deep recursion (about 10,000+ levels) throws `StackOverflowError`; convert to a loop or BFS.

3.9 Trees

A binary tree node has `val`, `left`, `right`. Most tree problems are solved by recursion: "answer for this node = combine answers of left and right". BFS (queue) handles level-based problems. In a **BST**, inorder traversal is sorted.

Node and height

PYTHON (what you know)

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = self.right = None

def height(root):
    return 0 if not root else 1 + max(height(root.left), height(root.right))
```

JAVA (what to write tomorrow)

```
class TreeNode {
    int val; TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

static int height(TreeNode root) {
    return root == null ? 0 : 1 + Math.max(height(root.left), height(root.right));
}
```

JAVA: tree templates

```
// Traversals (recursive)
static void inorder(TreeNode r, List<Integer> out) {    // left, node, right (sorted for BST)
    if (r == null) return;
    inorder(r.left, out); out.add(r.val); inorder(r.right, out);
}

static void preorder(TreeNode r, List<Integer> out) {    // node, left, right
    if (r == null) return;
    out.add(r.val); preorder(r.left, out); preorder(r.right, out);
}

static void postorder(TreeNode r, List<Integer> out) {    // left, right, node
    if (r == null) return;
    postorder(r.left, out); postorder(r.right, out); out.add(r.val);
}

// Iterative inorder with a stack
static List<Integer> inorderIter(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        cur = st.pop(); res.add(cur.val); cur = cur.right;
    }
    return res;
}

// Level order (BFS) -> list of levels
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int sz = q.size();                // nodes in this level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < sz; i++) {
            TreeNode n = q.poll();
            level.add(n.val);
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
        res.add(level);
    }
    return res;
}

static int countNodes(TreeNode r) { return r == null ? 0 : 1 + countNodes(r.left) +
    countNodes(r.right); }

static TreeNode invert(TreeNode r) {
    if (r == null) return null;
    TreeNode t = r.left; r.left = invert(r.right); r.right = invert(t);
    return r;
}

static boolean isSymmetric(TreeNode a, TreeNode b) {    // call with (root.left, root.right)
    if (a == null && b == null) return true;
    if (a == null || b == null || a.val != b.val) return false;
    return isSymmetric(a.left, b.right) && isSymmetric(a.right, b.left);
}
```

```

}

static boolean hasPathSum(TreeNode r, int target) {
    if (r == null) return false;
    if (r.left == null && r.right == null) return r.val == target;
    return hasPathSum(r.left, target - r.val) || hasPathSum(r.right, target - r.val);
}

static boolean isBalanced(TreeNode r) { return check(r) != -1; }
static int check(TreeNode r) { // returns height or -1 if unbalanced
    if (r == null) return 0;
    int l = check(r.left); if (l == -1) return -1;
    int rr = check(r.right); if (rr == -1) return -1;
    return Math.abs(l - rr) > 1 ? -1 : 1 + Math.max(l, rr);
}

static int diameter; // longest path in edges
static int depth(TreeNode r) {
    if (r == null) return 0;
    int l = depth(r.left), rr = depth(r.right);
    diameter = Math.max(diameter, l + rr);
    return 1 + Math.max(l, rr);
}

static TreeNode lca(TreeNode r, TreeNode p, TreeNode q) { // lowest common ancestor (any binary tree)
    if (r == null || r == p || r == q) return r;
    TreeNode L = lca(r.left, p, q), R = lca(r.right, p, q);
    if (L != null && R != null) return r;
    return L != null ? L : R;
}

// Validate BST: pass allowed (lo, hi) range down. Use long to survive Integer.MIN/MAX nodes.
static boolean isBST(TreeNode r, long lo, long hi) {
    if (r == null) return true;
    if (r.val <= lo || r.val >= hi) return false;
    return isBST(r.left, lo, r.val) && isBST(r.right, r.val, hi);
} // call: isBST(root, Long.MIN_VALUE, Long.MAX_VALUE)

// BST search / insert
static TreeNode search(TreeNode r, int key) {
    if (r == null || r.val == key) return r;
    return key < r.val ? search(r.left, key) : search(r.right, key);
}
static TreeNode insert(TreeNode r, int key) {
    if (r == null) return new TreeNode(key);
    if (key < r.val) r.left = insert(r.left, key); else r.right = insert(r.right, key);
    return r;
}

// Right side view = last node of every level (BFS) ; zigzag = reverse every other level
// Max path sum: like diameter but track max(0, child gain) and add node.val on both sides.

```

3.10 Graphs

Represent as an **adjacency list**. DFS (recursion/stack) explores deep; BFS (queue) explores by distance. Always keep a **visited** structure so you do not loop forever. For grids, treat each cell as a node with 4 neighbours.

Build adjacency list (nodes 0..n-1)

PYTHON (what you know)

```

graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)

```

JAVA (what to write tomorrow)

```

List<List<Integer>> g = new ArrayList<>();
for (int i = 0; i < n; i++) g.add(new ArrayList<>());
for (int[] e : edges) {
    g.get(e[0]).add(e[1]);
    g.get(e[1]).add(e[0]); // omit this line for a directed graph
}

```

JAVA: graph templates

```

static void dfs(List<List<Integer>> g, int u, boolean[] vis) {
    vis[u] = true;
    for (int v : g.get(u)) if (!vis[v]) dfs(g, v, vis);
}

```

```

static void bfs(List<List<Integer>> g, int src, int[] dist) { // dist[] = shortest steps from src
    Arrays.fill(dist, -1);
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(src); dist[src] = 0;
    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : g.get(u)) {
            if (dist[v] == -1) { dist[v] = dist[u] + 1; q.offer(v); }
        }
    }
}

static int countComponents(int n, List<List<Integer>> g) {
    boolean[] vis = new boolean[n]; int c = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) { c++; dfs(g, i, vis); }
    return c;
}

// Number of islands (grid DFS, flood fill)
static int numIslands(char[][] grid) {
    int cnt = 0;
    for (int i = 0; i < grid.length; i++)
        for (int j = 0; j < grid[0].length; j++)
            if (grid[i][j] == '1') { cnt++; sink(grid, i, j); }
    return cnt;
}

static void sink(char[][] g, int i, int j) {
    if (i < 0 || j < 0 || i >= g.length || j >= g[0].length || g[i][j] != '1') return;
    g[i][j] = '0';
    sink(g, i + 1, j); sink(g, i - 1, j); sink(g, i, j + 1); sink(g, i, j - 1);
}

// Shortest path in a grid (0 = open, 1 = wall) from top-left to bottom-right, BFS
static int shortestPath(int[][] g) {
    int r = g.length, c = g[0].length;
    if (g[0][0] == 1 || g[r - 1][c - 1] == 1) return -1;
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    boolean[][] seen = new boolean[r][c];
    Queue<int[]> q = new ArrayDeque<>();
    q.offer(new int[]{0, 0, 0}); seen[0][0] = true; // {row, col, steps}
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        if (cur[0] == r - 1 && cur[1] == c - 1) return cur[2];
        for (int[] d : dirs) {
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (nr >= 0 && nc >= 0 && nr < r && nc < c && !seen[nr][nc] && g[nr][nc] == 0) {
                seen[nr][nc] = true;
                q.offer(new int[]{nr, nc, cur[2] + 1});
            }
        }
    }
    return -1;
}

// Rotting oranges style: multi-source BFS = push ALL sources into the queue first, then BFS by levels.

// Topological sort / course schedule (Kahn's algorithm). Edge p[1] -> p[0].
static boolean canFinish(int n, int[][] pre) {
    List<List<Integer>> g = new ArrayList<>();
    int[] indeg = new int[n];
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] p : pre) { g.get(p[1]).add(p[0]); indeg[p[0]]++; }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
    int done = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); done++; // 'done' order is the topological order
        for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
    }
    return done == n; // false means a cycle exists
}

// Dijkstra (non-negative weights). g.get(u) holds {v, weight}.
static int[] dijkstra(List<List<int[]>> g, int src) {
    int n = g.size();
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE); dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {

```

```

        int[] cur = pq.poll(); int u = cur[0], d = cur[1];
        if (d > dist[u]) continue; // stale entry
        for (int[] e : g.get(u)) {
            int v = e[0], nd = d + e[1];
            if (nd < dist[v]) { dist[v] = nd; pq.offer(new int[]{v, nd}); }
        }
    }
    return dist;
}

// Union-Find (disjoint set): connectivity, cycle detection in undirected graph, Kruskal MST
static int[] parent;
static void initDSU(int n) { parent = new int[n]; for (int i = 0; i < n; i++) parent[i] = i; }
static int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
static boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false; // already connected -> this edge makes a
    cycle
    parent[ra] = rb; return true;
}

// Bipartite check (2-colouring) with BFS: color[] initialised to -1
static boolean bipartite(List<List<Integer>> g, int n) {
    int[] color = new int[n]; Arrays.fill(color, -1);
    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;
        Queue<Integer> q = new ArrayDeque<>(); q.offer(s); color[s] = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : g.get(u)) {
                if (color[v] == -1) { color[v] = 1 - color[u]; q.offer(v); }
                else if (color[v] == color[u]) return false;
            }
        }
    }
    return true;
}

```

Problem type	Use	Complexity
Reachability, components, flood fill	DFS or BFS	$O(V + E)$
Fewest steps, unweighted shortest path	BFS	$O(V + E)$
Weighted shortest path (no negatives)	Dijkstra + PriorityQueue	$O((V + E) \log V)$
Ordering with dependencies / cycle in directed graph	Topological sort (Kahn)	$O(V + E)$
Cycle in undirected graph, dynamic connectivity	Union-Find	about $O(E)$

3.11 Dynamic Programming Basics

DP = recursion with remembering. Steps: (1) define **dp[i]** in words, (2) write the recurrence (how **dp[i]** depends on earlier states), (3) set base cases, (4) choose the iteration order, (5) return the right cell. If you can write the recursion, add a memo array and you have top-down DP.

JAVA: DP templates

```

// 1D: climbing stairs (ways to reach step n using 1 or 2 steps)
static int climb(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}

// 1D: house robber (no two adjacent). prev = dp[i-2], cur = dp[i-1]
static int rob(int[] a) {
    int prev = 0, cur = 0;
    for (int x : a) { int nxt = Math.max(cur, prev + x); prev = cur; cur = nxt; }
    return cur;
}

// Unbounded: coin change (fewest coins), dp[a] = min coins for amount a
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // "infinity"
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)

```

```

        for (int c : coins) if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}

// Number of ways to make the amount (order does not matter): coins in OUTER loop
static int coinWays(int[] coins, int amount) {
    int[] dp = new int[amount + 1]; dp[0] = 1;
    for (int c : coins) for (int a = c; a <= amount; a++) dp[a] += dp[a - c];
    return dp[amount];
}

// 0/1 knapsack: iterate capacity BACKWARDS so each item is used once
static int knapsack(int[] wt, int[] val, int W) {
    int[] dp = new int[W + 1];
    for (int i = 0; i < wt.length; i++)
        for (int w = W; w >= wt[i]; w--) dp[w] = Math.max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}

// Longest increasing subsequence, O(n^2)
static int lis(int[] a) {
    int n = a.length, best = 1;
    int[] dp = new int[n]; Arrays.fill(dp, 1);
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) if (a[j] < a[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
        best = Math.max(best, dp[i]);
    }
    return best;
}

// 2D: longest common subsequence
static int lcs(String a, String b) {
    int n = a.length(), m = b.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                ? dp[i - 1][j - 1] + 1
                : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}

// 2D grid: unique paths (only right/down)
static int uniquePaths(int r, int c) {
    int[][] dp = new int[r][c];
    for (int i = 0; i < r; i++) for (int j = 0; j < c; j++)
        dp[i][j] = (i == 0 || j == 0) ? 1 : dp[i - 1][j] + dp[i][j - 1];
    return dp[r - 1][c - 1];
}

// Edit distance
static int editDistance(String a, String b) {
    int n = a.length(), m = b.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1) ? dp[i - 1][j - 1]
                : 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j - 1]));
    return dp[n][m];
}

```

DP family	State	Examples
Linear / 1D	<code>dp[i]</code> = best answer up to index <code>i</code>	Stairs, house robber, max subarray, LIS
Unbounded choice	<code>dp[amount]</code>	Coin change, rod cutting
0/1 choice	<code>dp[i][w]</code> or rolled to <code>dp[w]</code>	Knapsack, subset sum, partition equal sum
Two strings	<code>dp[i][j]</code> over prefixes	LCS, edit distance, distinct subsequences
Grid	<code>dp[r][c]</code>	Unique paths, min path sum

3.12 Greedy, Sorting and Intervals

JAVA: greedy, intervals, heap

```
// Merge overlapping intervals
```

```

static int[][] merge(int[][] iv) {
    Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    for (int[] cur : iv) {
        if (res.isEmpty() || res.get(res.size() - 1)[1] < cur[0]) res.add(cur);
        else res.get(res.size() - 1)[1] = Math.max(res.get(res.size() - 1)[1], cur[1]);
    }
    return res.toArray(new int[0][]);
}

// Minimum meeting rooms: sweep with two sorted arrays
static int minMeetingRooms(int[][] iv) {
    int n = iv.length; int[] s = new int[n], e = new int[n];
    for (int i = 0; i < n; i++) { s[i] = iv[i][0]; e[i] = iv[i][1]; }
    Arrays.sort(s); Arrays.sort(e);
    int rooms = 0, best = 0, j = 0;
    for (int i = 0; i < n; i++) {
        while (e[j] <= s[i]) { j++; rooms--; }
        rooms++; best = Math.max(best, rooms);
    }
    return best;
}

// Best time to buy and sell stock (one transaction)
static int maxProfit(int[] p) {
    int minPrice = Integer.MAX_VALUE, best = 0;
    for (int x : p) { minPrice = Math.min(minPrice, x); best = Math.max(best, x - minPrice); }
    return best;
}

// Jump game: can you reach the last index?
static boolean canJump(int[] a) {
    int far = 0;
    for (int i = 0; i < a.length; i++) { if (i > far) return false; far = Math.max(far, i + a[i]); }
    return true;
}

// Top K frequent elements with a min-heap of size k
static int[] topK(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : a) cnt.merge(x, 1, Integer::sum);
    PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>((x, y) -> x.getValue() - y.getValue());
    for (Map.Entry<Integer, Integer> e : cnt.entrySet()) { pq.offer(e); if (pq.size() > k) pq.poll(); }
    int[] res = new int[k];
    for (int i = k - 1; i >= 0; i--) res[i] = pq.poll().getKey();
    return res;
}

```

4. Practice Problems and Warm-ups

NOTE: These are representative problems at the level of a 20-minute screening round, grouped by pattern. They are not confirmed past questions. Solve at least one per pattern in Java before you sleep.

Pattern	Problems to practise	Key idea
Arrays	Max subarray, move zeroes, rotate array, missing number, second largest, majority element	Single pass, running variables, in-place swaps
Strings	Valid palindrome, anagram, reverse words, first unique char, longest common prefix, string compression	Two pointers, int[26], StringBuilder
HashMap/Set	Two sum, group anagrams, subarray sum = K, longest consecutive, duplicate check, isomorphic strings	Store what you have seen, prefix sums
Stack/Queue	Valid parentheses, next greater, daily temperatures, min stack, queue using stacks	Monotonic stack, push/pop pairing
Two pointers	3Sum, container with most water, remove duplicates, sort colors, trapping rain water	Sort first, move pointers by comparison
Sliding window	Longest unique substring, min size subarray sum, max sum of k, permutation in string	Expand right, shrink left while invalid
Binary search	Search insert position, first/last position, rotated array, Koko bananas, ship packages in D days	Monotonic check, lo/hi on answer range
Recursion	Subsets, permutations, combination sum, generate parentheses, N-Queens (hard)	Choose, explore, undo
Trees	Height, level order, symmetric, path sum, LCA, validate BST, diameter	Recursion returning info upward; BFS for levels
Graphs	Number of islands, rotting oranges, course schedule, clone graph, shortest path in grid	Visited set, BFS for distance, Kahn for ordering
DP	Climbing stairs, house robber, coin change, LIS, LCS, knapsack, unique paths	Define state, recurrence, base case
Matrix	Spiral order, rotate image, set matrix zeroes, search in sorted 2D matrix	Boundaries l/r/t/b, transpose + reverse

Classic warm-ups in Java

JAVA: frequently asked small problems

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; (long) i * i <= n; i++) if (n % i == 0) return false;
    return true;
}

static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
static int reverseInt(int n) { // 123 -> 321, returns 0 on overflow
    long rev = 0;
    while (n != 0) { rev = rev * 10 + n % 10; n /= 10; }
    return (rev > Integer.MAX_VALUE || rev < Integer.MIN_VALUE) ? 0 : (int) rev;
}

static boolean isPalindromeNumber(int n) {
    if (n < 0) return false;
    int orig = n, rev = 0;
    while (n > 0) { rev = rev * 10 + n % 10; n /= 10; } // may overflow for huge n; use long if needed
    return orig == rev;
}

static void fizzBuzz(int n) {
    for (int i = 1; i <= n; i++) {
        if (i % 15 == 0) System.out.println("FizzBuzz");
        else if (i % 3 == 0) System.out.println("Fizz");
        else if (i % 5 == 0) System.out.println("Buzz");
        else System.out.println(i);
    }
}

static int missingNumber(int[] a) { // numbers 0..n with one missing
    int n = a.length, sum = n * (n + 1) / 2;
    for (int x : a) sum -= x;
    return sum;
}

static int secondLargest(int[] a) {
```

```

    int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int x : a) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x < first) second = x;
    }
    return second;
}

static int majority(int[] a) { // Boyer-Moore voting
    int cand = 0, cnt = 0;
    for (int x : a) { if (cnt == 0) cand = x; cnt += (x == cand) ? 1 : -1; }
    return cand;
}

static int countVowels(String s) {
    int c = 0;
    for (char ch : s.toLowerCase().toCharArray()) if ("aeiou".indexOf(ch) >= 0) c++;
    return c;
}

static List<Integer> spiral(int[][] m) {
    List<Integer> res = new ArrayList<>();
    int t = 0, b = m.length - 1, l = 0, r = m[0].length - 1;
    while (t <= b && l <= r) {
        for (int j = l; j <= r; j++) res.add(m[t][j]); t++;
        for (int i = t; i <= b; i++) res.add(m[i][r]); r--;
        if (t <= b) { for (int j = r; j >= l; j--) res.add(m[b][j]); b--; }
        if (l <= r) { for (int i = b; i >= t; i--) res.add(m[i][l]); l++; }
    }
    return res;
}

static void rotateImage(int[][] m) { // 90 degrees clockwise: transpose, then
    reverse rows
    int n = m.length;
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) { int t = m[i][j]; m[i][j] = m[j][i];
        m[j][i] = t; }
    for (int[] row : m) for (int i = 0, j = n - 1; i < j; i++, j--) { int t = row[i]; row[i] = row[j];
        row[j] = t; }
}

```

A full 20-minute walkthrough

Problem: "Given a string, return the length of the longest substring without repeating characters."

Min	What you do
0-2	Examples: "abcabcbb" -> 3, "bbbb" -> 1, "" -> 0. Constraint: string length up to 10^5 , so $O(n^2)$ is too slow.
2-4	Brute force: check every substring, $O(n^3)$ or $O(n^2)$ with a set. Better: sliding window with a map of last seen index -> $O(n)$.
4-12	Write the method from section 3.5. Use <code>int[128]</code> for last positions. Name variables <code>left</code> , <code>right</code> , <code>best</code> .
12-17	Dry run "abba": at right=3 (a), last[a]=0 but left=2 already, so the <code>last[c] >= left</code> check matters. Without that condition you would move left backwards. This is the classic bug.
17-20	Check empty string, single char, all same, all distinct. Submit.

5. AI-Assisted Debugging Round (~20 min)

You get a program that compiles or runs but is wrong (or crashes). You can use an AI assistant. You are graded on **fixing the real bug quickly with a minimal change**, and on how well you use the AI. The AI is a fast pair-programmer, not an oracle: you must still read and verify.

5.1 Workflow (target: 20 minutes)

Step	Action	Time
1. Reproduce	Run it. Copy the exact failing input, expected output, actual output or stack trace.	2 min
2. Read fast	Read the problem description, then the main method, then helper methods. Skim for the data structure and the algorithm being attempted.	3 min
3. Classify	Compile error (syntax/type) vs runtime exception vs wrong answer (logic) vs timeout.	1 min
4. Hypothesise	Dry-run the smallest failing input by hand. Form 1-2 guesses for the cause.	3 min
5. Ask AI	Paste the relevant method + failing case + error + your hypothesis (prompts below).	2 min
6. Verify	Do NOT trust blindly. Check that the explanation matches your dry-run.	2 min
7. Minimal fix	Change the fewest lines possible. No rewrites, no renaming, no reformatting.	2 min
8. Test	Re-run the failing case, the sample, and 2-3 edge cases (empty, one element, duplicates, negative, large).	4 min
9. Submit	Make sure no debug prints remain and it still compiles.	1 min

5.2 Reading an unfamiliar codebase quickly

1. Find the entry point (`main`, the method named in the problem, the test file).
2. Read the method signatures and class names first, not the bodies.
3. Follow the data: where is the input created, transformed, and returned?
4. Look at the tests or sample I/O: they tell you the intended behaviour.
5. Search (Ctrl+F / Ctrl+Shift+F) for the variable or message that appears in the error.
6. Look for suspicious spots: loop bounds, `<` vs `<=`, index `+1/-1`, initial values, missing `return`, comparison operators, integer division.

5.3 Three kinds of errors

Kind	How it shows up	How to find it
Syntax / compile	Red error before running: missing <code>;</code> , <code>cannot find symbol</code> , <code>incompatible types</code> , <code>missing return statement</code>	Read the line number in the message; usually the line itself or the one just above
Runtime	Program crashes with an exception and a stack trace	Read the TOP line of the trace in YOUR code; the exception name tells the cause
Logical	Runs fine, output is wrong	Dry-run smallest failing input; print intermediate values; compare with expected step by step

Exception	Usual cause in DSA code
<code>ArrayIndexOutOfBoundsException</code>	Loop <code>i <= n</code> , accessing <code>a[i+1]</code> at the end, empty array, wrong 2D dimension
<code>NullPointerException</code>	Uninitialised object/array element, <code>map.get(k)</code> returned null then unboxed, tree/list node is null
<code>StringIndexOutOfBoundsException</code>	<code>charAt(len)</code> , bad <code>substring(i, j)</code> bounds
<code>NumberFormatException</code>	<code>Integer.parseInt</code> on empty/space-containing/non-numeric text
<code>StackOverflowError</code>	Missing or unreachable base case, or recursion depth too large
<code>ConcurrentModificationException</code>	Removing from a list/map while iterating with for-each
<code>ArithmeticException: / by zero</code>	Division or modulo by 0 (check divisor)
<code>NoSuchElementException</code>	<code>pop()/remove()</code> on empty deque, <code>Scanner</code> with no more input
<code>InputMismatchException</code>	<code>Scanner.nextInt()</code> but input is not an int

Exception	Usual cause in DSA code
<code>OutOfMemoryError</code> / Time Limit	Infinite loop, $O(n^2)$ on big input, creating objects in a loop

5.4 The common DSA bugs checklist

Symptom	Likely bug	Fix
Off by one, misses last element	<code>i < n - 1</code> vs <code>i < n</code> , <code><=</code> vs <code><</code>	Test with $n = 1$ and $n = 2$
Wrong for big input / negative output	<code>int</code> overflow in sum/product	Use <code>long</code> ; cast before multiplying: <code>(long) a * b</code>
Strings "equal" but condition false	<code>==</code> on Strings/Integers	Use <code>.equals()</code>
Infinite loop in binary search	<code>lo = mid</code> with <code>lo < hi</code> or <code>mid</code> rounds down	Use <code>lo = mid + 1</code> , or round mid up
Result list shows duplicates/empties (backtracking)	Added <code>cur</code> without copying, or forgot undo	<code>res.add(new ArrayList<>(cur))</code> , remove last after recursion
Works once, fails on second call	Static/global not reset	Reset static fields at the start of each call
<code>remove(i)</code> removed wrong thing	<code>List<Integer>.remove(int)</code> removes by index	<code>list.remove(Integer.valueOf(x))</code>
Wrong min/max for negative inputs	Initialised best to 0	Start with <code>Integer.MIN_VALUE</code> / first element
Graph search never ends	Missing visited, or marked visited too late	Mark visited when you ENQUEUE
Counts too high in graph/grid	Visited array not shared or reset wrongly	One visited array per full search
Sort result looks wrong	Comparator uses subtraction / wrong key / sorted copy ignored	<code>Integer.compare</code> ; sort the real array
Wrong with duplicates	Skipping or not skipping equal values	Add <code>if (i > start && a[i] == a[i-1]) continue;</code> after sorting
Integer division surprises	<code>a / b</code> where both int, then compared as decimal	Use <code>(double) a / b</code>
Mutating input	Function modifies the array the caller still needs	Copy first: <code>Arrays.copyOf</code>
Silent wrong answer after catch	Empty <code>catch</code> hides an exception	Never swallow exceptions while debugging

5.5 Worked debugging example

Example: binary search that hangs

```
// BUGGY: should return the index of target in a sorted array, or -1. Hangs on some inputs.
static int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid;      // BUG 1: lo never advances when hi = lo + 1
        else hi = mid;                    // BUG 2: should be mid - 1 for this loop style
    }
    return -1;                          // BUG 3: with lo < hi, a single remaining element is never
    checked
}

// FIX (minimal, consistent style): loop condition lo <= hi, lo = mid + 1, hi = mid - 1
static int searchFixed(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

5.6 How to prompt the AI for debugging

TIP: Give the AI the smallest useful context: the failing method, input, expected vs actual, and the constraint "change as few lines as possible". Ask it to explain the cause BEFORE giving code.

PROMPT TEMPLATE: debugging

I'm debugging a Java method. Do not rewrite it; find the root cause and propose the smallest fix.

Problem: <one-line description of what the method should do>

Language: Java 17

Code:

<paste only the relevant method(s)>

Failing case:

Input: <exact input>

Expected: <expected>

Actual: <actual output or full stack trace>

What I already checked: <your hypothesis / what you ruled out>

Please:

1. Explain the root cause in 2-3 sentences, pointing to the exact line.
2. Show the minimal change as a diff (only changed lines).
3. List 3 edge cases I should re-test after the fix.

Useful follow-up prompts

Quick follow-ups you can use:

- "Walk through the code line by line for input [2,1,3] and show variable values at each step."
- "Is the bug in the algorithm or in the implementation? Explain."
- "Give me a test case that would still fail after your fix, if any."
- "Is there any other bug in this method? List them without fixing."
- "Explain this stack trace and tell me which line of MY code to look at."

- **Do:** isolate one bug at a time; re-run after each change; keep the original code style.
- **Do not:** paste the whole project; accept a full rewrite; fix three things at once; skip testing because the AI sounded confident.
- **Tip:** If the AI answer does not match your dry-run, tell it your trace and ask it to reconcile.

6. AI-Assisted Feature Development (~45 min)

You receive an existing application and must add or change one feature. The test checks that you can **navigate a codebase, plan, implement incrementally, and verify**, using AI to go faster, not to replace thinking. The stack is whatever the platform provides (for example Spring Boot, Node/Express, React/Next.js). The method below works for all of them.

6.1 Time budget

Minutes	Phase	Output
0-5	Read the task. Run the app. Skim README and folder structure.	App runs; you know the entry points
5-12	Locate relevant files; trace one existing similar feature end to end.	List of files to touch (3-6 files)
12-17	Plan with AI: ask for an explanation and a step plan, not code.	Numbered checklist of small steps
17-38	Implement step by step, run after each step.	Working feature
38-43	Test normal case, edge cases, and that old features still work.	Verified behaviour
43-45	Clean up: remove console.log/prints, check git diff, save everything.	Clean, minimal diff

6.2 Understanding the project quickly

1. Open the root: **README**, **package.json** / **pom.xml** / **build.gradle**: tells you the stack, scripts, dependencies.
2. Start the app with the documented command. Click through the UI or call an endpoint to see current behaviour.
3. Look at the folder structure: where are routes/controllers, services, models, components, tests?
4. Find a feature similar to the one you must build. **Copy its pattern** for naming, layering and error handling.
5. Use global search for words from the task (for example "task", "priority", "filter") to find the right files.
6. Check the data model/schema/DTOs first: most features need a new field, validation, or query.

Stack	Typical request flow (where to look)
Spring Boot	Controller (@RestController , @GetMapping) -> Service (@Service) -> Repository (JpaRepository) -> Entity (@Entity) + DTOs. Config: application.properties .
Node / Express	routes/ -> controllers/ -> services or models/ (Mongoose/Sequelize/SQL). Entry: app.js / server.js . Middleware for auth/validation.
React	Page/component -> state (useState , context, Redux) -> API call (fetch /axios in api/ or services/) -> render. Props flow down.
Next.js	app/ or pages/ routes, app/api/.../route.ts handlers, server vs client components (" use client ").
Full-stack feature	DB/model change -> backend endpoint -> API client function -> UI component -> test in browser.

6.3 Use AI the right way

Good use	Bad use
"Explain what this controller/service does" (paste 1-2 files)	"Build the whole feature for me" with no context
"Given these files, propose a step-by-step plan to add X"	Pasting the entire repo
"Write only the new service method for X following this existing method's style"	Accepting code you did not read
"Why does this error occur?" with the real stack trace	Letting AI refactor unrelated files
"Write 5 test inputs, including edge cases, for this endpoint"	Adding new libraries/frameworks

PROMPT SEQUENCE: feature development

STEP 1 - UNDERSTAND (no code yet)

I'm working in an existing <Spring Boot | Express | React> app. Here are the relevant files: <paste 2-4 files>.

Task: <paste the feature description>.

Explain how data currently flows for <similar feature>, and which files I must change to implement the task.

Do not write code yet.

STEP 2 - PLAN

Propose a numbered plan of small steps (each testable on its own). Mention exact file names and function names.

Keep changes minimal and follow the existing code style. No new dependencies.

STEP 3 - IMPLEMENT ONE STEP AT A TIME

Implement step 1 only: <describe>. Show only the changed/new code with its file name.

Match the naming and error handling used in <existing file>.

STEP 4 - DEBUG / VERIFY

Step 1 compiled, but calling <endpoint/UI action> gives <error/behaviour>. Here is the log: <paste>.

What is the likely cause? Give the minimal fix.

STEP 5 - REVIEW

Here is my final diff: <paste>. Check for bugs, missing validation, null handling and edge cases.

Do not rewrite; list issues in priority order.

6.4 Worked example: "add priority to tasks and filter by it"

1. **Model:** add **priority** (enum or string, default **MEDIUM**) to the Task entity/schema/DTO.
2. **Persistence:** if using a DB with migrations or JPA auto-DDL, make sure the column is created; if a JSON/in-memory store, update seed data.
3. **Backend endpoint:** accept **priority** on create/update; add an optional query parameter **?priority=HIGH** on the list endpoint; validate allowed values.
4. **Frontend:** add a dropdown to the form, show a badge in the list, add a filter control that calls the API with the parameter.
5. **Test:** create one task of each priority; filter by each; submit with no priority (default applies); send an invalid value (should return a clear error); confirm old tasks still load.

WATCH OUT: Common ways candidates lose time: modifying files they did not understand, rewriting working code, not running the app until minute 35, and fixing UI before the API works. Always get the backend response right first, then connect the UI.

7. Prompt Engineering Round (~15-20 min)

You are given a problem statement and must **write the prompt** you would send to an AI to get a correct, useful result. You are graded on clarity, completeness, constraints and how well the prompt ensures the output can be checked. The framework from your plan:

Part	What to write	Example
Context	Role, background, tech stack, audience, existing code or data	"You are a senior Java developer reviewing a Spring Boot service used by 10k users."
Task	One clear action verb and the goal	"Write a method that returns the k most frequent words in a list."
Constraints	Language/version, complexity, libraries allowed/forbidden, style, length, edge cases to handle	"Java 17, no external libraries, $O(n \log k)$, handle ties alphabetically, null or empty input returns empty list."
Output format	Exactly what to return and in what structure	"Return: 1) approach in 3 bullets, 2) code in one block, 3) complexity, 4) table of 5 test cases."
Validation	How to verify; ask AI to self-check; request tests and edge cases	"Include unit tests (JUnit 5) and mentally run the sample input; list assumptions you made."

TIP: Memory line: **C-T-C-O-V**: Context, Task, Constraints, Output, Validation. Write them as labelled sections; graders can see each element at a glance.

7.1 Weak prompt vs strong prompt

Weak prompt	Why weak	Strong prompt (outline)
"Write code for sorting."	No language, algorithm, input, size, output, edge cases	Context: Java 17 backend. Task: implement merge sort for <code>int[]</code> . Constraints: $O(n \log n)$, stable, no <code>Arrays.sort</code> , handle empty/one-element and duplicates. Output: code + complexity + 3 tests. Validation: dry-run <code>[5, 2, 4, 2]</code> .
"Fix my code."	No code, error, or expected behaviour	Context: the method below should return the longest palindromic substring. It returns "" for "babad". Task: find root cause. Constraints: minimal change, keep signature. Output: explanation + diff. Validation: re-trace "babad" and "cbbd".
"Make a login page."	No stack, fields, validation, security, style	Context: React 18 + Tailwind, existing <code>api/auth.js</code> . Task: build <code>LoginForm</code> with email+password. Constraints: client-side validation, disabled button while loading, accessible labels, no new libraries. Output: one component file + usage example. Validation: list manual test steps.
"Explain recursion."	Unknown audience and depth	Context: explaining to a first-year student who knows loops. Task: explain recursion with factorial and Fibonacci. Constraints: under 250 words, one analogy, no jargon. Output: explanation + one trace table. Validation: end with 2 practice questions.

7.2 Ready-to-use templates

TEMPLATE: technical problem

[TECHNICAL PROBLEM PROMPT]

Context: You are an expert `<Java/Python/...>` engineer helping me prepare production-quality code.

Task: Solve the following problem: `<problem statement>`.

Input/Output: `<describe format and examples>`.

Constraints: `<language + version>`; `time <= O(...)`; `space <= O(...)`; `n up to <...>`; no external libraries;

handle `<edge cases: empty, duplicates, negatives, overflow>`.

Output format:

1. Brute-force idea and why it is too slow (2 lines)
2. Optimised approach (3-4 bullet points)
3. Complete code in a single code block with comments on key lines
4. Time and space complexity

Validation: Dry-run the sample input step by step, then provide 5 extra test cases including edge cases with

expected outputs. State any assumptions you made.

TEMPLATE: feature development

[FEATURE DEVELOPMENT PROMPT]

Context: Existing <stack> application. Relevant code: <paste files>. Conventions: <naming, layering, error handling>.

Task: Add <feature> so that <user-visible behaviour>.

Constraints: Do not change unrelated files; no new dependencies; keep existing API responses backward compatible;

validate input; return clear error messages.

Output: (1) short plan, (2) code changes grouped by file name, (3) how to run and manually test.

Validation: List happy path, 3 edge cases and 2 failure cases with expected results.

TEMPLATE: tests and review

[TEST GENERATION / REVIEW PROMPT]

Context: Below is a <language> function that <purpose>.

Task: Generate test cases that try to break it.

Constraints: Cover boundary values, empty input, duplicates, negatives, very large values, invalid types.

Output: A table with columns: input | expected output | what it tests. Then flag any input where the function's current behaviour looks wrong, with a one-line reason.

7.3 Improving a weak prompt (exercise style)

A likely task is "here is a poor prompt, rewrite it". Use this checklist and then write the improved version:

1. Is the **goal** stated with one clear verb?
2. Is the **context** given (role, tech, audience, existing code)?
3. Are **inputs and outputs** defined, with an example?
4. Are **constraints** listed (language, complexity, style, forbidden things, length)?
5. Is the **format** specified (sections, table, JSON, code block)?
6. Is there a **validation** request (tests, edge cases, self-check, assumptions)?
7. Is it **one task** (split if it asks for many unrelated things)?
8. Is it **unambiguous** (no pronouns with unclear referents like "it" or "that")?

Other techniques worth naming in an answer: give 1-2 examples (few-shot), ask the model to reason step by step before the final answer, set a role/persona, ask it to ask clarifying questions if requirements are missing, request a structured format (JSON/table) when the output will be parsed, and iterate in follow-up prompts rather than one huge prompt.

7.4 How to answer the round in 15-20 minutes

Min	Action
0-3	Read the problem statement. Underline: goal, audience, stack, any numbers or limits, required output.
3-5	Jot the five headings: Context, Task, Constraints, Output, Validation. Fill with bullet notes.
5-14	Write the prompt in full sentences under those headings. Be specific; include at least one example.
14-18	Re-read as if you were the AI: could anything be interpreted two ways? Add missing edge cases, format instructions and a validation step.
18-20	Final polish: remove fluff, fix spelling, make sure it is a single copy-paste-able prompt.

WATCH OUT: Do not write a prompt that asks the AI to "do everything" or "make it perfect". Also do not leave out the output format. Graders look for specifics they can check.

8. Final Checklist and Cheat Sheet

8.1 Before you start tomorrow

- Charge your laptop, stable internet, close extra tabs and apps; test camera/mic if proctored.
- Have water, ID if required, and a pen and paper for dry-runs (if allowed).
- Read each round's instructions fully, including language options and whether AI tools are enabled.
- Know the submission method: some platforms need "Run" and then "Submit"; unsaved code can be lost.
- If the editor allows choosing Java, select it first and check the class/method template provided.

8.2 Java compile-error survival

Error message	Meaning / fix
cannot find symbol	Misspelled variable/method, missing <code>import java.util.*;</code> , wrong case (<code>String</code> not <code>string</code>)
incompatible types: possible lossy conversion from long to int	Cast <code>(int)</code> or declare the variable as <code>long</code>
missing return statement	Some path through the method has no <code>return</code>
non-static method cannot be referenced from a static context	Make the helper <code>static</code> or create an object
variable might not have been initialized	Give it an initial value before the branch
generic array creation/unexpected type: int	Use <code>Integer</code> in generics: <code>List<Integer></code> , not <code>List<int></code>
class X is public, should be declared in a file named X.java	Class name must match the file/platform requirement (often <code>Main</code>)
unreported exception IOException	Add <code>throws IOException</code> to the method or use try/catch
array required, but ArrayList found	Use <code>list.get(i)</code> not <code>list[i]</code> ; <code>s.charAt(i)</code> not <code>s[i]</code>

8.3 One-page Python to Java reminder

Python	Java
<code>len(a), len(s), len(lst)</code>	<code>a.length, s.length(), lst.size()</code>
<code>a[-1]</code>	<code>a[a.length - 1]</code>
<code>a[i:j]</code>	<code>Arrays.copyOfRange(a, i, j)</code> or <code>s.substring(i, j)</code>
<code>range(n)</code>	<code>for (int i = 0; i < n; i++)</code>
<code>x // y, x % y, x ** y</code>	<code>x / y, x % y</code> (careful with negatives), <code>Math.pow(x, y)</code>
<code>True / False / None</code>	<code>true / false / null</code>
<code>and / or / not</code>	<code>&& / / !</code>
<code>elif</code>	<code>else if</code>
<code>for x in lst</code>	<code>for (int x : arr)</code> or <code>for (T x : list)</code>
<code>d.get(k, 0)</code>	<code>map.getOrDefault(k, 0)</code>
<code>k in d, x in s</code>	<code>map.containsKey(k), set.contains(x)</code>
<code>import heapq; heappush</code>	<code>PriorityQueue<Integer> pq; pq.offer(x)</code>
<code>collections.deque, append, popleft</code>	<code>ArrayDeque, offer / addLast, poll / pollFirst</code>
<code>float("inf")</code>	<code>Integer.MAX_VALUE</code> (or <code>Long.MAX_VALUE</code> , <code>Double.POSITIVE_INFINITY</code>)
<code>print(a, b)</code>	<code>System.out.println(a + " " + b)</code>
<code>sorted(a)</code>	<code>int[] b = a.clone(); Arrays.sort(b);</code>
<code>" ".join(words)</code>	<code>String.join(" ", words)</code>
<code>a, b = b, a</code>	<code>int t = a; a = b; b = t;</code>

Python	Java
<code>return a, b</code>	Return <code>int[]{a, b}</code> or a small class
<code>lambda x: ...</code>	<code>x -> ...</code>

8.4 Top 20 rules to remember

1. Read constraints first; they decide the algorithm.
2. Brute force first, optimise second; submit something that passes samples.
3. Use `long` for sums and products; `Integer.MAX_VALUE` is about 2.1 billion.
4. Strings: `equals`, not `==`. Build with `StringBuilder`.
5. Array length is `.length`, String is `.length()`, List is `.size()`.
6. `mid = lo + (hi - lo) / 2`; keep one binary-search style consistently.
7. Check `isEmpty()` before `pop/peek/poll`.
8. Backtracking: add a COPY of the list and UNDO after recursion.
9. Graphs/grids: mark visited when you enqueue; check bounds first in the `if`.
10. Trees: base case `null` first; think "what do I return to my parent?".
11. DP: define `dp[i]` in words before writing the loop.
12. Sort + two pointers handles most "pair/triple" problems with duplicates.
13. Sliding window needs non-negative numbers; otherwise prefix sums + map.
14. Use `Integer.compare` in comparators, not subtraction.
15. Dry-run the sample and one edge case before submitting.
16. Debugging: reproduce, isolate, minimal change, re-test. No rewrites.
17. Feature dev: understand first, plan, small steps, run after each step.
18. AI is a helper: give it small precise context, verify every answer.
19. Prompts: Context, Task, Constraints, Output, Validation.
20. Keep calm: partial credit exists; manage the clock per round.

NOTE: You already know the logic from Python. Tomorrow is mostly about translating cleanly: types, `.length` vs `.length()`, collections, and not overflowing. Good luck!